

**Intangible-Asset Risk Guardian (IARG) – Agentic Solution that provides asset
discovery, risk scoring and coverage optimization**

Project Workbook

By
Akshara Puligilla
Amrutha Junnuri
Chandini Saisri Uppuganti
Revanth Malladi

10/09/2025

Advisor: Prof. Gopinath Vinodh

Chapter 1. Literature Search, State of the Art

Literature Search

Research at the intersection of AI-driven asset intelligence, intangible asset valuation, and dynamic risk management demonstrates strong growth in both academic and industry domains. The literature indicates four major pillars that shape the foundation of this project.

(i) AI Risk Assessment Frameworks and Governance Standards

Recent research emphasizes standardized evaluation methods for the reliability and safety of AI based systems. Tong et al. present a first principles framework supporting the IEEE P3396 standard, which separates AI risk into process risk and outcome risk, establishing clear accountability across data flow and model operations [1]. The framework introduces an information centric approach to categorize AI outputs into perception, knowledge, decision, and control actions; each linked to measurable harms and mitigation mechanisms.

This principle directly supports the Intangible Asset Risk Guardian (IARG), providing a foundation for continuous monitoring of model behavior, data exposure, and valuation-based insurance triggers. By grounding its scoring layer in IEEE P3396 principles, IARG ensures traceability, fairness, and policy readiness across its architecture.

Sun et al. expand this foundation through a detailed taxonomy of AI failure propagation and cascading model risk in interconnected intelligent systems [2]. Their analysis identifies how compound errors can amplify across dependent models, emphasizing the need for causal traceability and transparent lineage mapping. These ideas directly inform IARG's asset graph engine, which captures and scores dependencies across AI assets to anticipate systemic risk. Together, these frameworks enable measurable, auditable, and policy aligned AI risk management.

(ii) Intangible Asset Discovery and Visibility in Enterprise Systems

The second pillar addresses the discovery and mapping of intangible assets within organizations. Li et al. propose a semantic graph learning framework for detecting and linking enterprise digital artifacts such as code, data, and models [3]. Their approach uses representation learning and graph embeddings to continuously update a registry of assets, improving organizational visibility and valuation accuracy.

This discovery mechanism aligns directly with IARG's asset graph module, which integrates repositories from GitHub, Google Drive, and Snowflake to build a unified inventory. IEEE Standard 3396 further strengthens this direction by prescribing governance schemas and metadata standards for documenting AI related assets and risks

[4]. IARG extends this standard by adding autonomous discovery agents that update asset states in real time and link discovered items to valuation and coverage models.

(iii) Valuation Models for Intangible and AI Assets

Keller and Liu propose a quantitative model for estimating the economic worth of intangible assets using neural network estimators that correlate innovation, software usage, and AI performance metrics with market value [5]. Their approach introduces learning based valuation proxies that can adapt dynamically to new inputs.

For IARG, this approach bridges the gap between technical risk data and monetary exposure. Each asset discovered in the system receives a dynamic value score that reflects both operational significance and vulnerability. This ensures that valuation directly informs insurance pricing, allowing real time adjustment of coverage thresholds and policy recommendations.

(iv) AI Based Risk Prediction and Coverage Optimization

Zhao et al. present a reinforcement learning framework that optimizes cyber insurance coverage by adapting premiums and protection terms according to real time threat data [6]. Their findings show that dynamic premium adjustment improves efficiency and cost effectiveness compared to fixed static coverage.

IARG builds on this idea by combining valuation, discovery, and risk prediction into an autonomous coverage optimization layer. When asset risk or value scores exceed a defined limit, IARG automatically requests quotes for micro or parametric insurance policies and prepares evidence for claim readiness. This transforms insurance from a reactive process into proactive and continuous protection guided by live data streams.

Together, these four research pillars - AI governance [1][2], asset discovery [3][4], valuation [5], and coverage optimization [6] provide a comprehensive scholarly foundation for IARG. Each pillar corresponds to a core module within the system, contributing to a cohesive, value aligned, and data driven framework for intangible asset protection.

State of the Art Summary

The current state of the art in AI based asset intelligence and risk management is defined by a convergence of autonomous monitoring, AI governance, and insurance innovation.

Enterprise platforms such as Microsoft Defender for Cloud Apps, Google Cloud Digital Risk Protection, and Exabeam Fusion provide visibility across repositories and cloud networks. However, these focus primarily on anomaly detection and compliance alerts rather than linking asset value to dynamic coverage. None currently construct a complete registry of intangible assets or automatically trigger insurance policies.

In the insurance industry, leaders like Aon, Swiss Re, and Marsh are exploring data driven coverage models such as parametric and micro insurance for digital assets. These policies trigger based on quantifiable indicators such as breach scale or downtime duration. Meanwhile, IEEE P3396 and related governance initiatives define the vocabulary for accountability and harm measurement in AI systems, but implementations that directly link these frameworks to coverage workflows remain rare.

The Intangible Asset Risk Guardian advances this field by unifying asset discovery, valuation, and dynamic policy orchestration into a single agentic workflow. Its architecture introduces three major contributions:

1. Real time asset graph discovery across multiple enterprise sources.
2. Continuous risk and value scoring aligned with IEEE P3396 taxonomy.
3. Automated coverage orchestration using policy triggers informed by live telemetry.

Through this integration, IARG transforms static monitoring into active protection and closes the coverage gap between intangible asset value and existing insurance products. It represents a step toward economically resilient AI adoption and comprehensive intangible asset safeguarding.

References

1. Tong, R., Cortes, M., DeFalco, J. A., Underwood, M., and Zalewski, J. (2025). A First Principles Based Risk Assessment Framework and the IEEE P3396 Standard. Retrieved from <https://arxiv.org/pdf/2504.00091>
[This paper introduces the IEEE P3396 aligned framework for structured AI risk assessment and classification.]
2. Mohammad Hamad, (2023) Sebastian Steinhorst . Security Challenges in Autonomous Systems Design. Retrieved from <https://arxiv.org/html/2312.00018v2>
[This paper details cascading model risk and emphasizes traceability in multi agent AI environments.]
3. Jenifer Paulraj, Brindha Raghuraman, Nagarani Gopalakrishnan, Yazan Otoum (2025). *Autonomous AI-based Cybersecurity Framework for Critical Infrastructure: Real-Time Threat Mitigation*. Retrieved from <https://arxiv.org/abs/2507.07416>
[This paper proposes a hybrid AI-driven cybersecurity framework that enhances real-time vulnerability detection, threat modeling, and automated remediation for critical infrastructure systems such as energy grids, healthcare, and transportation networks.]
4. Mark Underwood, Christy Bahn (IEEE Computer Society). *AI-RSTR - Framework for Defining and Evaluating AI Risk, Safety, Trustworthiness and Responsibility*. Retrieved from <https://standards.ieee.org/ieee/3396/11379/>
[This IEEE working group under the Artificial Intelligence Standards Committee focuses

on establishing a standardized framework for assessing AI systems' risk, safety, trust, and ethical responsibility.]

5. Shaswata Mitra, Trisha Chakraborty, Subash Neupane, Aritran Piplai, Sudip Mittal (2024). *Use of Graph Neural Networks in Aiding Defensive Cyber Operations*. Retrieved from <https://arxiv.org/html/2401.05680v1>
[This paper explores how Graph Neural Networks (GNNs) can enhance defensive cybersecurity operations by improving threat detection, situational awareness, and adaptive response in complex cyber environments.]

Chapter 2. Project Justification

Most of a company's value today comes from intangible assets such as software, proprietary data, AI models, intellectual property, and brand reputation. Recent studies show that these assets make up nearly 90% of the total market value of S&P 500 companies. Despite their enormous value, insurance protection for these digital assets remains weak compared to traditional physical assets like buildings and equipment. This lack of protection leaves companies open to data theft, misuse, AI attacks, and cloud breaches, leading to financial and reputational loss.

This problem is becoming more critical as cyber threats grow in both number and complexity. Modern risks such as AI model attacks, data leaks, and major cloud breaches are becoming more common and costly. While some cyber insurers and technology providers offer monitoring tools, these solutions rarely connect asset valuation with dynamic insurance coverage. As a result, companies often struggle to recover quickly after incidents due to a lack of clear asset valuation and prepared claims evidence. This results in a serious lack of protection at a time when intangible assets hold most of the business value.

Several existing solutions attempt to address parts of the problem but fall short of providing a complete answer. For example, Google's Facade system, Microsoft Purview, Splunk, Exabeam, and Proofpoint offer monitoring and anomaly detection to reduce insider and external risks. Similarly, Google Cloud provides attack surface management and cyber risk protection services. However, these platforms mainly focus on detection and posture management. They do not build a full registry of intangible assets, assign defensible value to them, or automatically connect risk signals to insurance coverage. This limitation means that while companies may detect threats, they still lack the financial resilience and protection needed when attacks occur.

The Intangible-Asset Risk Guardian (IARG) aims to resolve this challenge by integrating asset discovery, valuation, risk assessment, and coverage optimization into a single platform. IARG continuously maps assets across different repositories assigning value based on international standards that are extended for AI models. It monitors risks such as data theft, membership inference, and large-scale breaches, and when thresholds are crossed, it automatically triggers insurance requests for micro and parametric policies. In addition, it prepares claims evidence in advance, allowing faster recovery and reduced downtime when incidents occur.

The significance of this project lies in its ability to surpass traditional monitoring by unifying asset valuation, risk assessment, and insurance coverage. IARG improves protection by matching security and insurance to the real value of intangible assets. It also makes AI use safer by handling risks tied to data and models. By protecting intangible assets, IARG can create lasting impact for both companies and the insurance sector.

Chapter 3. Project Requirements

3.1 Requirements

3.1.1. Essential Features

Use Case 1: Automated Asset Discovery

Use Case ID	UC-01
Use Case Name	Automated Asset Discovery
Primary Actor(s)	Security Officer (User), System (AI Model)
Goal	Automatically discover, classify, and register all intangible enterprise assets across multiple repositories.
Preconditions	1. User has valid access tokens for GitHub, HuggingFace, and Snowflake. 2. API permissions are granted for read-only crawling.
Main Flow of Events	1. User logs in to the dashboard and connects repositories (via OAuth). 2. System verifies tokens and establishes secure API sessions. 3. System crawls repository metadata (file names, extensions, commit history, owners, timestamps). 4. AI classifier identifies file types (source code, datasets, AI models, documents) using content-type recognition and keyword heuristics. 5. Dependencies between files are extracted using metadata analysis and stored as graph relationships. 6. Each asset entry is assigned a unique Asset ID, classification, and criticality level (low, medium, high). 7. System stores all records in the Enterprise Asset Registry (Neo4j DB). 8. System stores cached data in Redis.
Alternate Flow	1. If token expires → system prompts re-authentication. 2. If repository access fails → system logs error and continues with other sources.

Postconditions	1. Asset registry populated with all discovered assets, their types, and dependency graph. 2. Visualization dashboard displays discovered assets with filtering options.
Data Sources	GitHub API, HuggingFace API, Snowflake metadata catalog.
Output Artifacts	Asset Registry JSON entries, Visualization Graph.

Use Case 2: Dynamic Risk Scoring

Use Case ID	UC-02
Use Case Name	Dynamic Risk Scoring
Primary Actor(s)	Risk Analyst (User), System (Model)
Goal	Continuously compute and update risk scores for each asset based on live telemetry and vulnerability data.
Preconditions	1. Asset metadata and lineage already stored in registry. 2. Threat intelligence API (MITRE, CVE, etc.) connected. 3. Access frequency logs are available using Kafka.
Main Flow of Events	1. Risk Analyst logs into dashboard and selects one or more assets. 2. System retrieves metadata: owner, last modified date, access count, external exposure (e.g., public repo). 3. System fetches vulnerability indicators (e.g., CVEs, leaked secrets, outdated libraries). 4. Model computes risk using formula: Risk = (Exposure × Frequency × Vulnerability Weight) + AI-specific Risk Factors (model inversion, data leakage, data corruption). 5. Composite risk score normalized between 0–100. 6. Dashboard updates with color codes: • Low (0–30) • Medium (31–70) • High (71–100). 7. Scores auto-refresh daily or when new commits detected.
Alternate Flow	1. Missing metadata → system skips scoring for that asset. 2. Threat API timeout → system uses cached data.

Postconditions	1. Risk scores stored in registry with timestamp. 2. Dashboard visually ranks assets by priority for remediation.
Data Sources	Asset Registry, Threat Intelligence API, Access Logs.
Output Artifacts	Updated Risk Dashboard, Risk JSON Reports.

Use Case 3: Asset Valuation

Use Case ID	UC-03
Use Case Name	Asset Valuation
Primary Actor(s)	Insurance Manager (User), System (Model)
Goal	Estimate each intangible asset's fair economic value using standardized valuation models.
Preconditions	1. Registry populated with assets. 2. Risk scores and usage analytics available.
Main Flow of Events	1. Insurance Manager selects assets from dashboard for valuation. 2. Backend retrieves required data: asset type, usage frequency, business dependency, and risk score. 3. Applies IVS 210 and WIPO-aligned formula: For Source Code: Value = (Revenue Contribution × Usage Rate) ÷ (1 + Risk Factor) For Dataset: Value = (Market Reference Price × Quality Score) ÷ (1 + Risk Factor) For AI Model: Value = (Training Cost + Compute Hours + Fine-tuning Adjustments) × (1 – Depreciation Rate) ÷ (1 + Risk Factor) 4. System generates valuation reports with justifications (metrics, sources, timestamp). 5. Reports stored in valuation module and linked to each asset.

Alternate Flow	1. Missing risk score → default moderate risk factor (0.5). 2. API failure → retry once before fallback.
Postconditions	1. Every asset has a fair-value estimate with traceable metrics. 2. Reports are accessible for audit or insurance submission.
Data Sources	Asset Registry, Risk Module, Usage Analytics DB.
Output Artifacts	Valuation Report, Audit Report (PDF/JSON).

Use Case 4: Coverage Optimization

Use Case ID	UC-04
Use Case Name	Coverage Optimization
Primary Actor(s)	Business Executive (User), System (Model), External Insurance API
Goal	Automatically trigger and optimize insurance coverage when risk crosses thresholds.
Preconditions	1. Insurance API credentials stored securely. 2. Risk scoring system operational. 3. Coverage thresholds configured (e.g., 70/100).
Main Flow of Events	1. System continuously monitors risk scores. 2. When score > threshold (e.g., 70), system triggers insurance workflow. 3. Fetches real-time micro or parametric policy offers from insurance API (). 4. Presents top 3 coverage options with cost-benefit summaries. 5. Business Executive selects a preferred plan. 6. System sends policy selection to insurance API and links the policy ID to the asset record.
Alternate Flow	1. API down → retry after 30 mins. 2. No available offers → log warning and notify user.

Postconditions	1. Updated asset registry with policy details and coverage status. 2. Dashboard reflects active/inactive coverage per asset.
Data Sources	Risk Module, Insurance APIs.
Output Artifacts	Policy Records, Coverage Dashboard.

Use Case 5: Evidence Preparation and Claims Support

Use Case ID	UC-05
Use Case Name	Evidence Preparation and Claims Support
Primary Actor(s)	Claims Officer (User), System (Model)
Goal	Automatically assemble claim-ready evidence packages for incidents involving intangible asset losses.
Preconditions	1. Incident log or breach alert exists. 2. Relevant access logs and anomaly alerts are stored in the CloudWatch.
Main Flow of Events	1. Claims Officer selects an incident from the incident dashboard. 2. Backend gathers evidence: access logs (user IDs, timestamps), anomaly alerts (from SIEM), asset lineage (from registry), and risk history. 3. System correlates event timeline (incident → access → anomaly). 4. Generates structured evidence package (JSON/PDF) including: • Incident summary • Impacted assets • Risk evolution chart • Access anomalies and user activity logs • Timestamped system actions 5. Package stored in Claims Evidence Repository like S3. 6. Notification sent to insurer API (like Lloyd Sandbox) for review.

Alternate Flow	1. Missing data source → system flags incomplete evidence and alerts officer. 2. API delay → asynchronous retry queue.
Postconditions	1. Structured evidence package available for insurer audit. 2. Reduces claim processing time and dispute rate.
Data Sources	SIEM logs, Asset Registry, Risk Module, Incident Database.
Output Artifacts	Evidence Package (JSON/PDF), Claim Summary Dashboard.

3.1.2 Desired Features (Efficiency & Usability)

1. **Predictive Analytics:** Forecast emerging risks and asset degradation trends.
2. **Simulation Mode:** Enable “what-if” testing for coverage optimization.
3. **Natural Language Query Assistant:** Allow users to ask asset or risk questions conversationally.
4. **Cross-Platform Integration:** Support for Slack, Jira, and model registries.
5. **Multi-Language Dashboard:** Provide localization for international deployment.

3.1.3 Optional Features (Automation & Integrations)

1. **Blockchain Integration:** Record immutable audit logs for asset provenance and claims verification.
2. **Gamified Visualization:** Display risk through interactive heatmaps and dynamic graphs.
3. **Policy Recommendation Engine:** Suggest custom policy tiers based on enterprise exposure profile.
4. **External Marketplace Linkage:** Connect to third-party asset valuation or reinsurance platforms.

3.2 Non-Functional Requirements

1. **Performance:** The system must discover and score the assets with in few minutes.
2. **Scalability:** Must handle the assets across multiple repositories and concurrent users like SAAS or software based companies.
3. **Security & Privacy:** End-to-end encryption (TLS in transit, AES-256 at rest); GDPR and CCPA compliance.
4. **Reliability:** System uptime $\geq 99.9\%$ with fault-tolerant failover for monitoring and coverage orchestration.
5. **Usability:** Simple, intuitive dashboard usable by both technical and non-technical stakeholders.
6. **Auditability:** All scoring, valuation, and coverage actions must be immutably logged for compliance.
7. **Interoperability:** Integration via RESTful APIs and open standards (JSON, FHIR, ISO 27001 alignment).

3.3 Baseline Approaches

Existing platforms such as Microsoft Purview, Splunk UEBA, and Google Cloud Risk Protection provide detection and monitoring but lack comprehensive coverage of intangible asset valuation and dynamic insurance orchestration. The baseline industry approach therefore emphasizes monitoring and detection, whereas the IARG extends beyond by integrating:

- Asset graph discovery with valuation logic.
- Risk scoring directly tied to economic value.
- Dynamic micro and parametric insurance binding based on real-time risk thresholds.

Through this integrated approach, IARG exceeds current capabilities by creating a closed-loop ecosystem connecting asset intelligence, valuation, and coverage optimization within one agentic platform.

Chapter 4. Dependencies and Deliverables

Dependencies

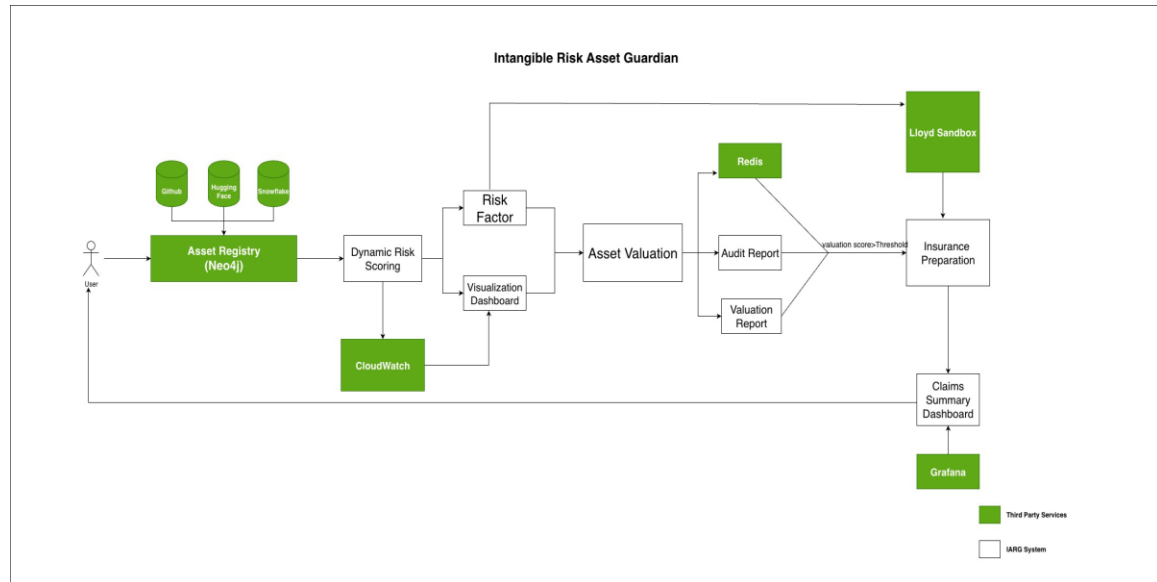
1. **Third-party APIs/Log Access:** if GitHub/Drive/MLflow/Snowflake audit APIs change, break, or have rate limits, event ingestion could stop.
2. **Cloud Infrastructure:** backend relies on cloud queues (Kafka/Redis) and storage; outages could halt risk scoring or actions.
3. **Synthetic Data Realism:** project evaluation depends on generating realistic insider-risk scenarios, if test data is too shallow, results won't be credible.
4. **Permission Scope:** without sufficient admin rights, the system may not be able to revoke links, rotate keys, or enforce actions.
5. **Integration Security:** weak auth/configuration in connectors could itself become a vulnerability.
6. **Storage Limits:** limited storage capacity for large-scale deployment.

Deliverables

1. **Working Prototype** of the Guardian system with connectors for GitHub and one non-repo source, e.g., Google Drive or model registry.
2. **Synthetic Test Dataset** and evaluation results demonstrating detection latency, accuracy, and exposure-hour reduction.

Chapter 5. Project Architecture

Project Architecture



System Overview

This diagram illustrates an automated system for real-time asset risk valuation and insurance preparation. Its primary goal is to dynamically assess the risk associated with digital assets, determine their value based on that risk, and then use this data to automate the process of securing insurance coverage through a provider like Lloyd's.

The workflow can be broken down into five main stages:

1. Asset Registration and Management

- **Components:** Asset Registry (Neo4j), External Databases, User/Operator.
- **Function:** This is the starting point where all company assets are cataloged. Using a Neo4j graph database, the system stores information about assets and, crucially, the relationships between them. Data is populated into this registry from other company databases and potentially managed by a human operator.

2. Dynamic Risk Scoring

- **Components:** Dynamic Risk Scoring, CloudWatch, Visualization Dashboard.
- **Function:** The system pulls asset information from the registry and combines it with real-time operational data from CloudWatch (e.g., logs, access patterns,

performance metrics). It then calculates a live risk score for the assets. This score is visualized on a dashboard for immediate monitoring.

3. Asset Valuation and Auditing

- **Components:** Asset Valuation, Audit Report, Valuation Report.
- **Function:** The calculated Risk Factor is fed into the Asset Valuation engine. This component determines the current value of an asset, adjusted for its risk level. The output is a formal Audit Report and a Valuation Report, which serve as the official record of the asset's state and value at that moment.

4. Automated Insurance Preparation

- **Components:** Insurance Preparation, Lloyd Sandbox, Redis.
- **Function:** The Audit Report (containing a valuation score) is sent to the Insurance Preparation module. This module formats the data and communicates with an external insurance provider, represented here by the **Lloyd Sandbox**.
- **Redis** acts as a high-speed cache or messaging layer, facilitating quick data exchange between the internal valuation system and the external insurance platform.

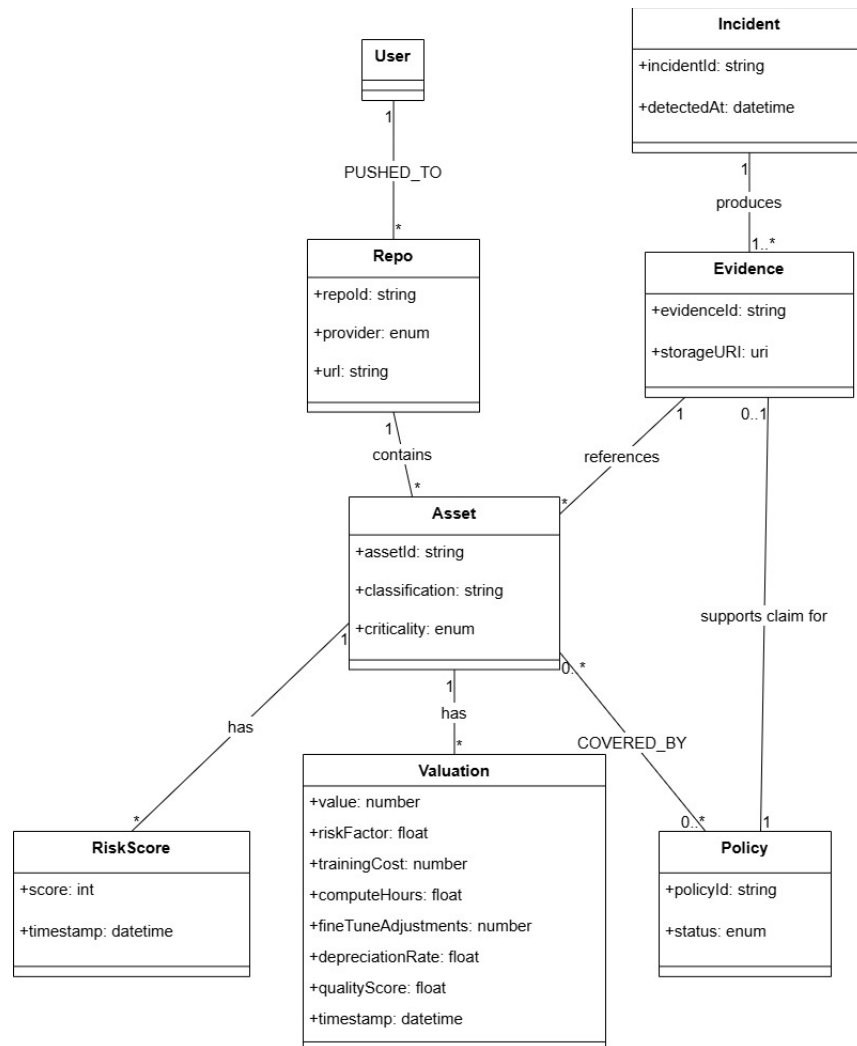
5. Monitoring and Reporting

- **Components:** Claims Summary Dashboard, Grafana.
- **Function:** The entire process is monitored through a **Grafana** dashboard. This Claims Summary Dashboard provides visibility into the status of insurance preparations and any subsequent claims. There is a feedback loop from this dashboard back to the operator, allowing for complete oversight of the system's performance and outcomes.

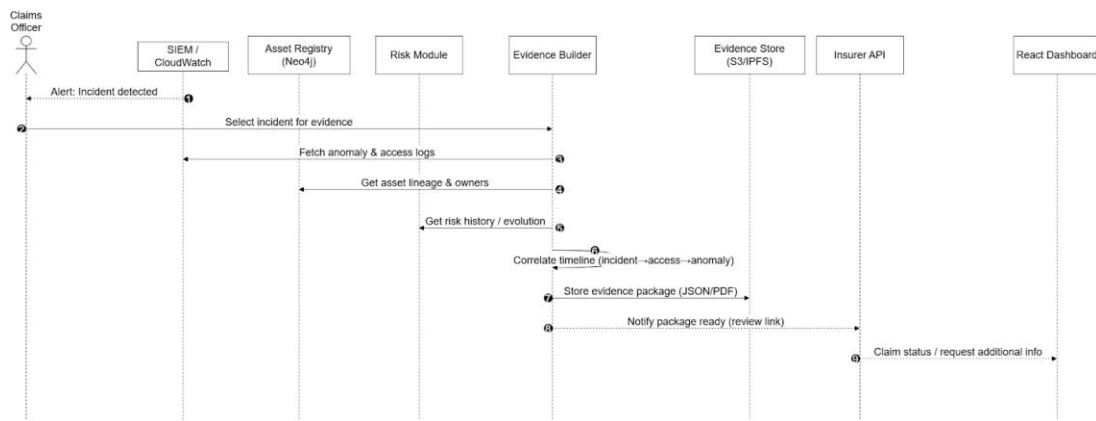
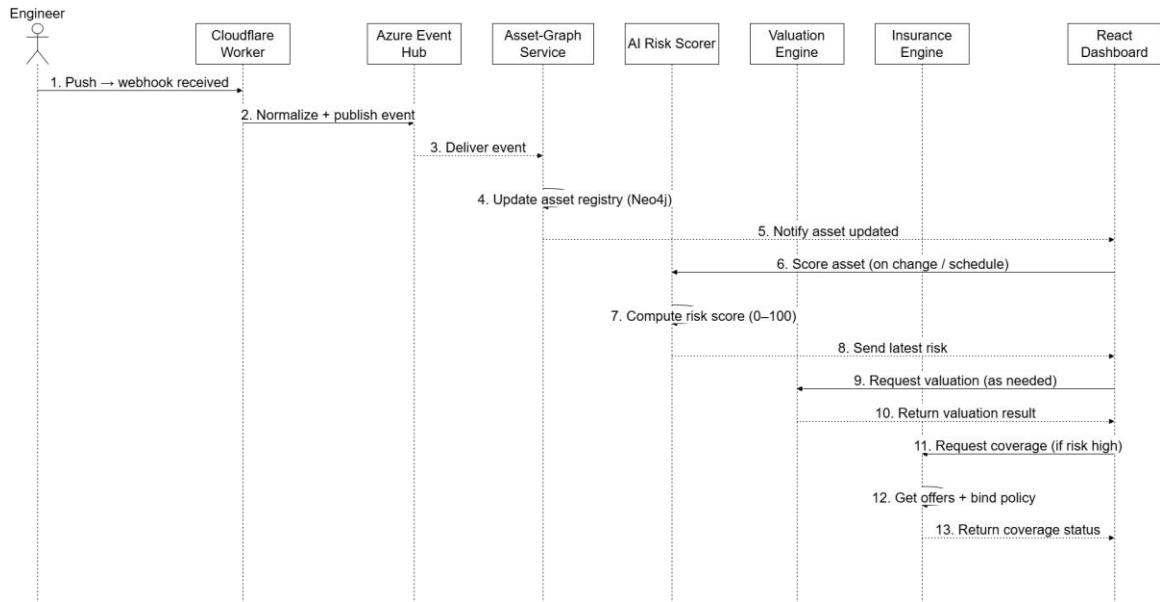
Chapter 6. Project Design

UML DIAGRAMS

UML Class Diagram



UML Sequence Diagrams



UML Component Diagram



Overview

Analytics

Assets

Incidents

Policies

Evidence

Graph

Settings

Search

KPI Title
123

KPI Title
123

KPI Title
123

KPI Title
123

Risk Trend (24h)

Top Risky Assets

Live Event Feed

Overview

Analytics

Assets

Incidents

Policies

Evidence

Graph

Settings

Search

KPI Title
123

KPI Title
123

KPI Title
123

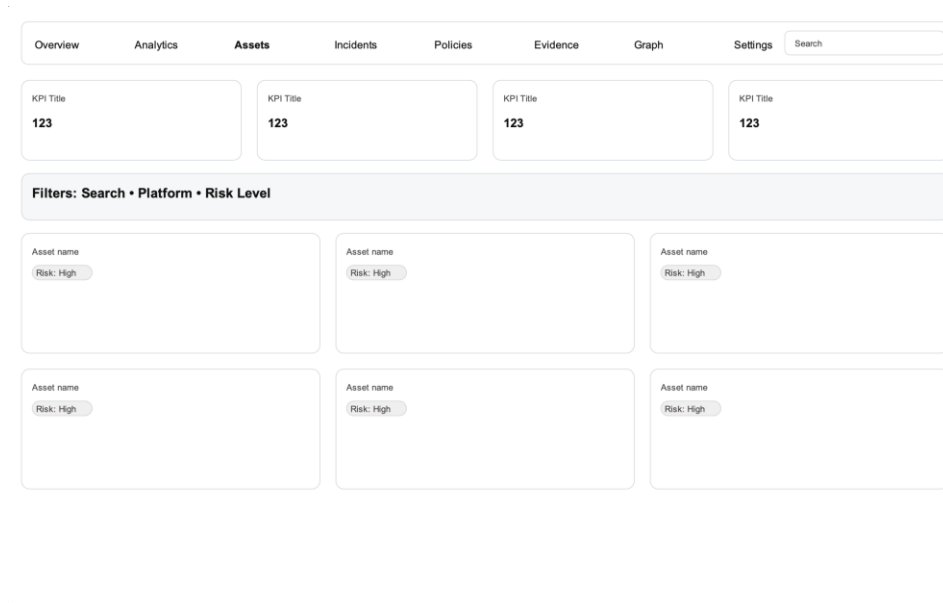
KPI Title
123

Risk Score Trends (24h)

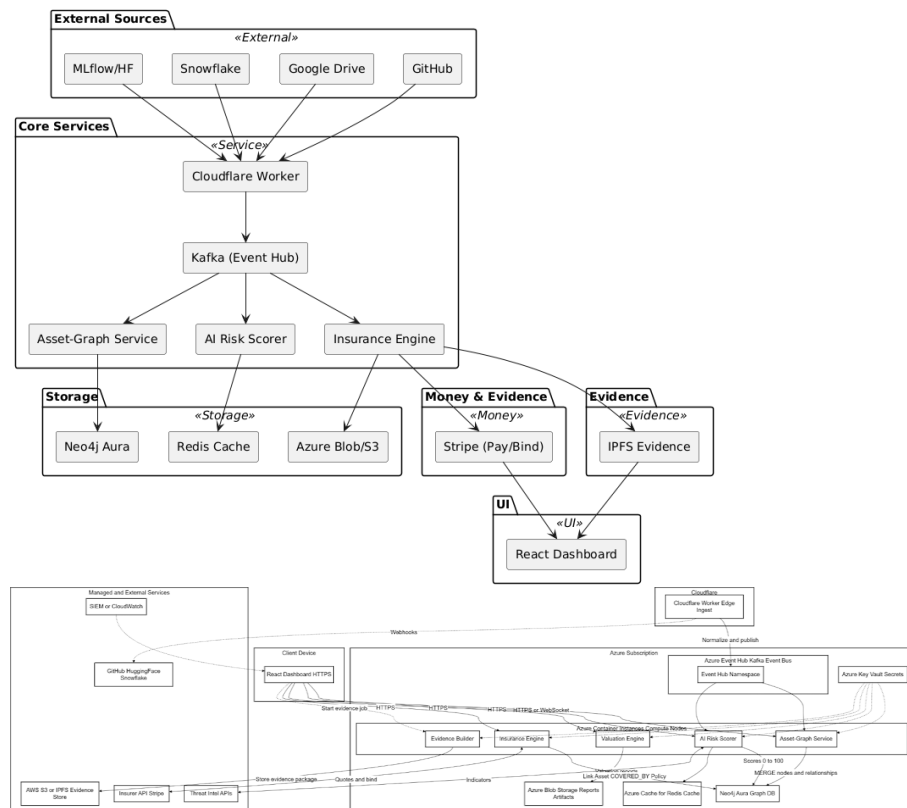
Platform Distribution

Risk Distribution

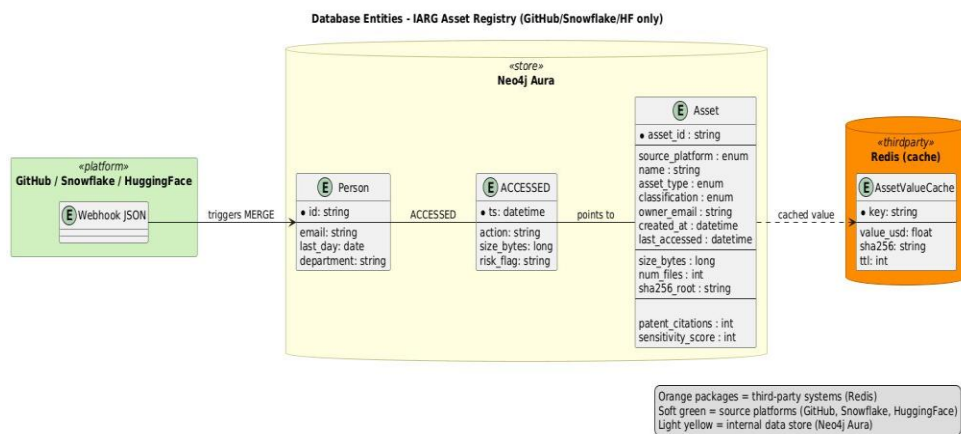
Event Activity



UML Component Diagram



Database Entity diagram



UI Mockup: <https://v0-iarg-web-app.vercel.app/>

Chapter 7. QA, Performance, Deployment Plan

7.1 Quality Assurance (QA) Plan

7.1.1 Objective

The goal of Quality Assurance (QA) in the Intangible-Asset Risk Guardian (IARG) project is to ensure that all modules, from asset discovery to coverage optimization perform reliably, accurately, and securely. QA activities aim to verify that the system meets its functional requirements, performance targets, and non-functional standards such as scalability, usability, and compliance.

7.1.2 QA Approach

IARG follows an iterative testing approach aligned with its microservice-based architecture. Each component is tested independently and then integrated into the full event-driven system. Continuous Integration (CI) ensures that every code change passes automated testing before deployment.

Test Level	Purpose	Description / Examples	Tools / Frameworks
Unit Testing	Validate individual components and logic functions.	Risk score computation, API endpoint behavior, and graph relationship creation.	PyTest (Python), Go Test, Vitest/Jest (React).
Integration Testing	Ensure smooth data flow and message exchange between services.	Kafka → Neo4j → Risk Scorer → Insurance Engine → Dashboard.	Postman, Docker Compose, Mock APIs.
Functional Testing	Confirm use case completion (UC-01 to UC-05).	Asset discovery, valuation, insurance triggering, and claims preparation.	Selenium, Playwright.

Security Testing	Identify vulnerabilities in APIs and data access.	OAuth validation, token expiry handling, data encryption verification.	OWASP ZAP, Azure Security Center.
Performance Testing	Evaluate speed, scalability, and throughput under load.	Stress testing the event-driven architecture with 1,000+ concurrent events.	Locust, JMeter.
User Acceptance Testing (UAT)	Validate overall usability and functionality with stakeholders.	Dashboard usability, visualization clarity, policy trigger verification.	Manual testing, stakeholder feedback forms.

7.1.3 QA Environment

Testing will be performed in three environments:

1. **Local (Docker Compose)** – Developer testing and debugging using simulated events.
2. **Staging (Azure)** – Integrated testing environment using Terraform-deployed infrastructure.
3. **Production (Azure Cloud)** – Final environment with monitoring, scaling, and logging enabled.

All environments use identical configurations to ensure test consistency.

7.1.3 Continuous Integration / Continuous Deployment (CI/CD)

- a. **Pipeline:** GitHub Actions → Lint → Unit Test → Integration Test → Build Docker Image → Push to Azure Container Registry → Deploy via Terraform.
- b. **Quality Gates:** Minimum 90% unit test coverage, All CI checks pass before merge, Static analysis (Flake8, ESLint) mandatory.

7.2 Performance Evaluation Plan

7.2.1 Performance Metrics

Metric	Target	Description
Asset Discovery Latency	≤ 60 seconds	Time from event ingestion to registry update.
Risk Scoring Time	≤ 3 seconds per asset	AI scoring model response time.
System Throughput	≥ 500 events/sec	Message processing capacity via Kafka/Event Hub.
Dashboard Refresh Rate	≤ 2 seconds	Real-time dashboard update interval.
Uptime / Availability	$\geq 99.9\%$	Ensured by redundant Azure containers and failover policies.

7.2.2 Performance Testing Scenarios

1. **Stress Test:** Simulate 10x load spikes on Event Hub to test autoscaling behavior.
2. **Soak Test:** Continuous event processing over 24 hours to measure memory and CPU stability.
3. **Failover Test:** Simulate Azure service failure and verify recovery via Terraform redeploy.
4. **Network Latency Simulation:** Introduce artificial delay to ensure resilience of asynchronous event queues.

7.2.3 Optimization Methods

- Use **Redis caching** to reduce risk-score recalculation.
- Optimize **Neo4j queries** with indexes on AssetID and Owner.
- Employ **parallel microservice execution** for scoring and valuation.
- Enable **serverless autoscaling** on Azure Container Instances for efficiency.

7.3 Deployment Plan

7.3.1 Infrastructure Setup

Layer	Technology	Purpose
Event Ingestion	Cloudflare Workers + Azure Event Hub	Capture and publish events securely.
Processing Layer	Azure Container Instances (Go, Python)	Handle graph updates, scoring, insurance automation.
Data Layer	Neo4j Aura, Redis, IPFS, Azure Blob	Persist assets, risk data, and evidence.
Frontend	React + Vite + Tailwind	Visualize assets, risk scores, and policies.
IaC & Automation	Terraform, GitHub Actions	Provision and manage cloud infrastructure.

7.3.2 Deployment Steps

- Provision Resources:**
Use Terraform to deploy Azure Resource Group, Event Hub, Key Vault, Blob Storage, and Container Instances.
- Deploy Services:**
Push Docker images from GitHub Actions to Azure Container Registry → Run containers.
- Configure Secrets & Access:**
Load credentials (GitHub, Insurance API, Redis) from Azure Key Vault.
- Validate Services:**
Run integration tests to confirm Event → Graph → Scoring → Policy chain.

5. Monitor & Log:

Grafana dashboards + Azure Monitor + CloudWatch for continuous performance and error tracking.

7.3.3 Rollback & Recovery

- **Versioned Deployments:** Each deployment tagged in GitHub and Terraform.
- **Automatic Rollback:** If health checks fail, system reverts to previous stable container image.
- **Data Backups:** Scheduled Neo4j and Blob backups every 12 hours.

7.4 Deliverables for QA and Deployment

- QA Test Plan Document
- CI/CD Configuration (GitHub Actions YAML)
- Performance Evaluation Report (Latency, Throughput, Accuracy)
- Terraform Deployment Scripts
- Grafana Monitoring Dashboard
- Deployment Verification Log

7.5 Algorithms

This section describes the detailed algorithms that define the functional logic of the Intangible-Asset Risk Guardian (IARG) system. Each algorithm includes its purpose, inputs, outputs, pseudocode, failure-handling approach, observability features, and testing methods.

7.5.1 Edge Event Ingestion (Cloudflare Worker to Azure Event Hub)

Purpose

To normalize incoming repository or webhook events and publish them reliably to the messaging bus.

Inputs

Hypertext Transfer Protocol (HTTP) request containing a vendor signature and payload data from sources such as GitHub, Hugging Face, or Snowflake.

Outputs

An event message titled *ingest.event.v1* published on Azure Event Hub or Apache Kafka.

Algorithm (Pseudocode)

```
onRequest(request):
  verifyHMAC(request.headers['X-Signature'], request.body)
  event = parseVendorEvent(request)
  normalized = {
```

```

    id: uuid_v7(),
    vendor: event.vendor,
    type: event.type,
    subject: event.subject,
    actor: event.actor,
    asset_hint: inferAssetType(event),
    timestamp: now_iso(),
    raw_reference: storeInBlobIfLarge(event.raw)
  }
  publish('ingest.event.v1', normalized)
  logMetric('ingest_success', 1, {vendor: event.vendor})

```

Failure Handling

Invalid signatures are rejected. Exponential backoff is applied for event publishing with a maximum of three retry attempts.

Observability

Each event includes a unique trace identifier that is propagated through the *traceparent* header for monitoring.

Testing Methods

- Unit Testing: verifies the signature authentication logic.
- Integration Testing: confirms that a POST request creates a valid event message on Azure Event Hub.
- Security Testing: ensures protection against signature spoofing.

7.5.2 Asset Discovery Crawler

Purpose

To crawl connected data sources, classify discovered digital assets, and persist structured information in the asset registry.

Inputs

OAuth authentication tokens and a list of source systems.

Outputs

Asset nodes and relationships stored in Neo4j, cached summaries in Redis, and a new event titled *asset.discovered.v1*.

Algorithm (Pseudocode)

```

crawl():
  for source in sources:
    token = getToken(source)

```

```

pages = getPaginatedList(source)
for item in pages:
    metadata = fetchMetadata(item)
    asset_type = classify(metadata)
    asset_id = stableHash(source, item.id)
    upsertNeo4j(asset_id, asset_type, metadata)
    cacheRedis(asset_id, summarize(metadata))
    publish('asset.discovered.v1', {asset_id, asset_type, owner: metadata.owner})

```

Failure Handling

If authentication tokens expire, they are refreshed automatically. When rate limits are reached, the system waits for the “Retry-After” interval and queues pending jobs.

Testing Methods

- Unit Testing: verifies the classification logic.
- Integration Testing: validates the pipeline between Kafka and Neo4j.
- Performance Testing: ensures that ten thousand artifacts are processed within sixty seconds.

7.5.3 Asset Graph Builder

Purpose

To maintain dependency and lineage relationships among all discovered assets.

Inputs

Events *asset.discovered.v1* and *ingest.event.v1*.

Outputs

Asset dependency edges stored in Neo4j and a new event named *graph.updated.v1*.

Algorithm (Pseudocode)

```

onEvent(message):
    transaction = beginTransaction()
    MERGE (a:Asset {assetId: message.assetId})
    for dependency in extractDependencies(message):
        MERGE (b:Asset {assetId: dependency.id})
        MERGE (a)-[:DEPENDS_ON {type: dependency.type}]->(b)
    commit(transaction)
    publish('graph.updated.v1', {root: message.assetId, count: dependencyCount})

```

Failure Handling

Transient database errors are retried with randomized waiting intervals. Failed messages are moved to a dead-letter queue for later inspection.

Testing Methods

- Unit Testing: validates Cypher query generation.
- Integration Testing: ensures that data flow from Kafka to Neo4j is reliable.
- Security Testing: verifies protection from query injection.

7.5.4 Dynamic Risk Scoring

Purpose

To compute a composite numerical risk score for each asset in near real time based on operational and contextual factors.

Inputs

Asset metadata, access frequency, threat intelligence data, and model-specific risk factors.

Outputs

A risk score event *risk.score.v1* and an updated *riskScore* property in the database.

Algorithm (Pseudocode)

```
onTrigger(asset_id):
  metadata = getAssetMetadata(asset_id)
  threat_data = getThreatIntel(asset_id)
  frequency = getAccessRate(asset_id, last_24h)
  ai_factors = computeAIRisk(metadata)
  raw_risk = (metadata.exposure * frequency * threat_data.weight) + ai_factors
  risk = scaleAndClamp(raw_risk, 0, 100)
  risk_band = assignRiskBand(risk)
  writeBack(asset_id, {risk, risk_band, timestamp: now})
  publish('risk.score.v1', {asset_id, risk, risk_band})
```

Performance Target

Processing time for each asset must not exceed three seconds.

Testing Methods

- Unit Testing: confirms accurate mathematical computation.
- Integration Testing: ensures the dashboard updates upon receiving score events.
- Performance Testing: measures latency during high-load event streaming.

7.5.5 Asset Valuation

Purpose

To calculate the fair economic value of each intangible asset using internationally recognized valuation standards.

Inputs

Asset type, usage statistics, dependency centrality, and current risk score.

Outputs

A valuation event *valuation.completed.v1* and a recorded value field in the registry.

Algorithm (Pseudocode)

```

if asset.type == "code":
    value = (revenue_contribution * usage_rate) / (1 + risk_factor)
elif asset.type == "dataset":
    value = (market_reference_price * quality_score) / (1 + risk_factor)
elif asset.type == "model":
    value = (training_cost + compute_hours + fine_tuning_adjustment) * (1 -
depreciation_rate) / (1 + risk_factor)
storeValuation(asset.id, value)
publish('valuation.completed.v1', {asset_id: asset.id, value: value})

```

Testing Methods

- Unit Testing: validates the accuracy of the valuation formulas.
- Integration Testing: confirms that valuation results link properly to asset entries.
- User Acceptance Testing: ensures that all valuation reports are complete and readable.

7.5.6 Coverage Optimization and Policy Binding

Purpose

To automatically initiate insurance coverage when an asset's risk score surpasses the defined threshold and to bind the selected policy.

Inputs

Risk score events, valuation results, and credentials for the external insurance provider.

Outputs

A policy binding confirmation event *policy.bound.v1*.

Algorithm (Pseudocode)

```

onRiskUpdate(asset_id, risk):
    if risk.score < threshold:
        return
    offers = fetchInsuranceOffers(asset_id, value, risk)
    ranked = rankByNetBenefit(offers)
    notifyDashboard(ranked[:3])
    if userSelectsOption(option):
        policy = bindPolicy(option.id)
        linkPolicyToAsset(asset_id, policy.id)
        publish('policy.bound.v1', {asset_id: asset_id, policy_id: policy.id})

```

Failure Handling

If the insurance provider interface is unavailable, retries occur every thirty minutes until successful.

Testing Methods

- Functional Testing: validates the end-to-end insurance binding workflow.
- User Acceptance Testing: confirms the accuracy of the cost-benefit presentation.
- Security Testing: verifies that all credentials are securely stored and retrieved from Azure Key Vault.

7.5.7 Evidence Packaging and Claims Support

Purpose

To assemble a claim-ready evidence package after a security incident and automatically notify the insurance provider.

Inputs

Incident logs, Security Information and Event Management data, asset lineage, and historical risk records.

Outputs

Evidence files in JavaScript Object Notation (JSON) or Portable Document Format (PDF) stored in Azure Blob Storage or the InterPlanetary File System (IPFS).

Algorithm (Pseudocode)

```

prepareEvidence(incident_id):
    incident = fetchIncident(incident_id)
    logs = querySIEM(incident.window, incident.assets)
    lineage = getLineage(incident.assets)
    risk_history = getRiskTimeline(incident.assets, incident.window)
    package = buildPackage(incident, logs, lineage, risk_history)
    uri = storePackage(package)

```

```
publish('claims.evidence.ready.v1', {incident_id, uri})  
notifyInsurer(uri)
```

Failure Handling

If required data is missing, the report is marked incomplete and the responsible officer is alerted. Delayed insurer responses are queued for retry.

Testing Methods

- Functional Testing: validates that evidence packages are generated for each incident.
- User Acceptance Testing: ensures clarity and readability of the evidence packet.
- Performance Testing: verifies stable behavior during heavy-load simulations.

Chapter 8. Implementation Plan and Progress

8.1 Project Vision and Scope

Build a real-time, event-driven risk intelligence platform that maps software activities, analyzes risks using AI, and automates insurance through a scalable, serverless cloud architecture integrating GitHub, Azure Event Hub, Neo4j, Stripe, and AI-driven scoring with Terraform-based automation and containerized microservices.

Primary Stack: Azure, Terraform, Go, Python, XGBoost, TensorFlow Serving, Neo4j Aura, Redis, IPFS, Azure Blob, AWS S3, React, Vite, Cloudflare Workers, Kafka, Azure Event Hub, Stripe API

8.2 Programming & Execution Environment

Languages

Python, Node.js, JavaScript, HTML5, CSS3, Terraform, SQL (Neo4j Cypher), Go

Local developer environment

- OS: macOS/Linux/Windows (WSL2)
- Package managers: pnpm (monorepo), nvm (Node versions).

/app

- edge/ (Cloudflare Worker for webhook ingestion)
- ingest/ (Event normalization and routing)
- graph/ (Go service for Neo4j asset graph)
- score/ (Python AI risk scorer (XGBoost, TF Serving))
- insure/ (Go insurance engine with Stripe integration)
- dash/ (React + Vite frontend dashboard)
- iac/ (Terraform IaC for Azure deployment)
- tools/ (Dev utilities, scripts, and CI/CD helpers)
- docs/ (Documentation and architecture diagrams)
- scripts/ (Local setup, health checks, and automation)

Runtime & process model

- **Event-Driven & Serverless:** Each microservice runs as a stateless Azure Container Instance, triggered by real-time events from Azure Event Hub (Kafka) and scaling to zero when idle.

- **Distributed Processing:** Cloudflare Workers handle edge ingestion, while Graph, Scorer, and Insure services run in parallel, updating the React + Vite dashboard in real time.

Security & access

- **Secure Access Control:** Secrets and credentials are stored in Azure Key Vault, with role-based access control (RBAC) and least-privilege principles applied across all services.
- **Data & Network Security:** All inter-service communication uses HTTPS and signed webhooks (HMAC), ensuring encrypted data flow between Cloudflare Workers, Event Hub, and backend containers.

Status: Planning complete.

Next Work: Begin implementation of cloud infrastructure using Terraform provisioning Azure Resource Group, Event Hub, Container Registry, and Key Vault as the project's foundational setup.

8.3 Development Tools Acquisition

Core tooling

- **Build/Dev:** Vite (dashboard), Go run (graph/insure), Python venv + Flask/FastAPI (scorer).
- **Style/Quality:** Black, Flake8, ESLint, Prettier, Husky + lint-staged, Commitlint + Conventional Commits.
- **Testing:** Pytest + Coverage (AI scorer), Go test (graph/insure), Vitest/Jest + React Testing Library (dashboard).
- **API Design:** OpenAPI/Swagger (FastAPI & Go Gin), Pydantic DTOs.
- **CI/CD:** GitHub Actions (lint → test → build → containerize → deploy).
- **Containers:** Docker, Docker Compose for local service stack (Kafka, Neo4j, Redis, IPFS).
- **Design System:** Tailwind CSS + Headless UI (dashboard).

AI adapters (Node)

- LangChain, OpenAI SDK, Hugging Face Hub, TensorFlow.js, ONNX Runtime, Pinecone, Weaviate, Redis Vector, Supabase Edge Functions, OpenAI Functions, Vercel AI SDK, LlamaIndex, Replicate, Cohere SDK, Anthropic SDK, Stability SDK, ElevenLabs API, Whisper API.

8.4 Prototyping Environment

Local stack (docker-compose)

- Kafka - (event streaming bus).
- neo4j - community (graph database for asset mapping).
- Redis - (cache + message queue).
- ipfs/kubo - (immutable object storage for evidence).
- minio (S3-compatible local storage for model artifacts).
- mock services: Stripe test API, Event simulation (JSON server).

Cloud staging (later)

- Azure Event Hub (Kafka-compatible message bus), Neo4j Aura (managed graph DB), Azure Cache for Redis, Azure Blob Storage (artifacts & policy archives), and Azure Container Instances (microservices).

Status: Local prototype planned.

Next: Finalized docker-compose.yml, add health-check scripts, and deploy Terraform IaC for staging environment.

8.5 Prototype-of-a-Prototype (P0)

Objective:

Build a minimal end-to-end pipeline that connects GitHub webhook ingestion to the event-driven architecture touching the Edge Worker, Event Hub, Graph service, AI Scorer, Insurance Engine, and Dashboard to validate full data and event flow.

P0 User Story:

“Trigger a GitHub push → Cloudflare Worker captures and validates event → Event published to Azure Event Hub → Graph service creates User–Repo link in Neo4j → AI Scorer assigns a risk score → Insurance Engine generates mock Stripe policy → Dashboard updates in real time.”

Scope & Constraints:

- Edge ingestion through Cloudflare Worker with HMAC validation and normalized event schema.
- Graph updates written to Neo4j via Go driver using MERGE queries for node and relationship creation.

- Risk score generation using a lightweight XGBoost model running in the Python scorer service.
- Policy issuance simulated via mock Stripe API; evidence pinned locally through IPFS.
- Real-time updates streamed via WebSocket to the React + Vite dashboard.

P0 Definition of Done:

- **API:** /ingest, /graph/update, /score/event, /policy/generate, /health.
- **DB:** Neo4j Aura (nodes: User, Repo, Policy; relationships: PUSHED_TO, COVERED_BY).
- **Web:** “Event Stream,” “Risk Gauge,” and “Active Policies” pages in React dashboard.
- **Tests:** Unit tests for Go and Python services; integration tests for full Kafka → Neo4j → Stripe flow.
- **CI:** GitHub Actions green on push; build, lint, and container tests pass; docker-compose up runs full stack locally.

Status: Planned (**currently working on webhook with Github, Google Drive**)

Next: Implement in **Phases 3-4**, starting with Event Hub → Neo4j pipeline validation, followed by AI scoring integration and dashboard streaming updates.

8.6. Project Plan

Category	Task	Owner	Status
Phase 1 – Research & Foundations	Project Research & Background Study	Revanth, Amrutha	Completed
Phase 1 – Research & Foundations	Literature Review & Problem Framing	Chandini, Akshara	Completed
Phase 1 – Research & Foundations	Requirement Analysis & Use Case Definition	Akshara, Revanth	Completed
Phase 1 – Research & Foundations (Output)	Finalized scope, functional and non-functional requirements, problem statement	Entire Team	Completed
Phase 2 – Technical Architecture & Stack Setup	Technology Stack Research (Go, Python, Azure, Neo4j, Kafka, Terraform)	Chandini, Amrutha	Completed

Phase 2 – Technical Architecture & Stack Setup	System Architecture & Data Flow Design (event-driven model)	Chandini, Revanth	Completed
Phase 2 – Technical Architecture & Stack Setup (Output)	Approved architecture diagram, finalized stack, Terraform templates drafted	Revanth, Chandini	Completed
Phase 3 – Data & Event Infrastructure	Synthetic Dataset Planning & Generation	Amrutha, Akshara	Planned
Phase 3 – Data & Event Infrastructure	Event Ingestion & Kafka (Azure Event Hub) Integration	Revanth, Chandini	Planned
Phase 3 – Data & Event Infrastructure (Output)	Local event stream simulation and verified message publishing across services	Revanth, Chandini	Planned
Phase 4 – Core Service Development	Graph Construction Service (Neo4j Integration)	Chandini	Planned
Phase 4 – Core Service Development	Risk Scoring Simulation (AI model baseline)	Amrutha	Planned
Phase 4 – Core Service Development (Output)	Verified graph updates and initial scoring functionality	Chandini, Amrutha	Planned
Phase 5 – Integration & Validation	Data Flow Validation and Log Analysis across services	Amrutha, Revanth	Planned
Phase 5 – Integration & Validation	Stripe mock integration and dashboard update pipeline	Akshara	Planned
Phase 5 – Integration & Validation (Output)	Full end-to-end data pipeline verified with real-time visualization	Entire Team	Planned
Phase 6 – Evaluation & Optimization	Risk Scoring Simulation and Performance Evaluation	Amrutha	Planned
Phase 6 – Evaluation & Optimization	Model accuracy testing, latency measurements, and resilience checks	Amrutha, Revanth	Planned
Phase 6 – Evaluation & Optimization (Output)	Validated test report demonstrating reliable model performance and optimized pipeline	Entire Team	Planned

Chapter 9. Project Schedule

